

Data Structures and Algorithms

Sheet #6

Searching, Hashing and Sorting

A. Searching

1. Using sequential search, search for the following keys: 20, 15, 21 and 31 in the following list.

Show the comparison order and number of comparisons.

20	25	19	22	15	10	30	21
----	----	----	----	----	----	----	----

2. Using binary search, search for the following keys: 26, 41, 23, 30, 5, and 50 in the following list. Show the comparison order and number of comparisons. At each loop iteration, including the last, show the contents of first, last, and mid.
3. Show the best and worst case of the following algorithms:
- Sequential Search
 - Sentinel Search
 - Binary Search
 - Ordered List Sequential Search
4. Write the recursive algorithm of binary search.

B. Hashing

1. Using the hash function $h(\text{key}) = \text{key} \bmod 11$,
- Apply the following collision resolution techniques to insert the following records.

30, 20, 56, 75, 31, 19, 22

--	--	--	--	--	--	--

- separate chaining
- Linear probing
- Quadratic probing
- Pseudorandom method with $a = 3$ and $b = 5$.

- For each technique of the above answer these questions:

- What is the load factor of this hash table?
- What is the number of comparisons needed to search for: 30, 31, 41, 19 and 44?

2. Using the modulo-division method and linear probing, store the keys shown below in an array with 19 elements. How many collisions occurred? What is the load factor of the hash table?

224562	137456	214562	140145	214576	162145	144467	199645	234534
--------	--------	--------	--------	--------	--------	--------	--------	--------

C. Sorting

1. For the following data, apply the following algorithms to sort data in **ascending** order.

Note: For all algorithms **the first part sorted is that on the LEFT**.

Show the result and number of inversions **after each pass**.

- a. Selection sort
- b. Insertion sort
- c. Bubble sort
- d. Shell sort

5	8	3	10	1	7	4	6	2	9
---	---	---	----	---	---	---	---	---	---

2. Repeat question 1 algorithms to sort data in **descending** order.
3. Repeat question 1 but with make the first part sorted is that on the **RIGHT**.
4. Repeat question 2 but make the first part sorted is that on the **RIGHT**.
5. For the following data, apply the following algorithms to sort data in **ascending** order.
Show result after each pass.

- a. Heap

- b. Quick (assume a pass is **one recursive call** (working on one pivot per pass) and the pivot element of a range is the **first element** on the left of that range)

index	0	1	2	3	4	5	6	7	8	9
Initial	21	50	14	19	30	17	40	27	12	13

6. What is the suitable algorithm if:
- a. the array is nearly sorted
 - b. the data is too large to fit in memory
7. What does it mean the sorting algorithm is stable? Give an example when stability really matters.
8. What is the best and worst case complexity of all of the above algorithms? Give examples.
9. Write the following algorithms to sort an array in **ascending** order and the first part sorted is that on the **LEFT**.
- a. Selection
 - b. Insertion
 - c. Bubble
 - d. Quick
 - e. Heap
10. Repeat question 9 to sort an array in **descending** order.
11. Repeat question 9 but make the first part sorted is that on the **RIGHT**.
12. Repeat question 10 but make the first part sorted is that on the **RIGHT**.